Agentic RAG - Documentation Updates

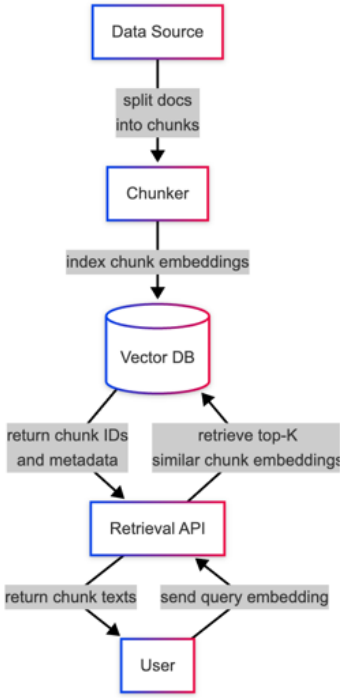
Purpose

In order to produce a Production grade system when it comes to retrieving relevant snippets of Documentation needed to answer a user's question effectively, we will need to expand the current state of our code form leveraging simple RAG to using some Agentic RAG capabilities. This document will outline the necessary refactoring's and features that will need to be included in order to make this work.

Long Term Architecture Vision

1. User asks a question
2. Decompose users question into multiple sub queries
3. We determine the Projects and corresponding Spaces that are impacted by the question
4. We then go through and retrieve relevant information from the Chroma DB associated with "relevant" Projects
5.

Current State - RAG



1. User sends a prompt
2. Our system will decompose the users question into sub-queries and determine *which ChromaDB collection* (**documentation, code, or both)** this sub-query can best be answered by
3. Using these decomposed queries, we will retrieve the **chunks** that are ***semantically similar*** to these sub-queries
4. After gathering all chunks, we'll simply utilize a CrossEncoder and compare the contents of the retrieved chunks to the ***user's original query,*** only taking the **top-5** chunks in terms of relevance score

Current State - Ingestion

NOTE: This logic is currently being done in separate worker threads and a *separate Database Transaction.* This is problematic as down-stream, this Ingestion Job may fail, but the ***File's*** in our database will be updated with latest information, causing for subsequent retries to think *we already processed each file correctly and no changes have been made since last time*

1. We recursively download files from the specified DataSource
2. Upon downloading the file, we will determine the file's status
   a. **CHANGED:** We simply indicate we need to re-ingest this file
   b. **UNCHANGED:** Skip re-ingesting this file

    c. **NEW:** Insert into our database and ingest file
3. Split the files into **documentation** & **code** files, which will influence how we go about **chunking the file**
    a. **CODE:** We split code files by **file extension** (i.e Python, Java, etc) and then leverage `CodeSplitter` from `llama_index` to chunk these files effectively
    b. **DOCS:** We chunk files utilizing `Docling`, by first converting temporary files downloaded to `Docling` files, chunking them, and then manually converting them to `llama_index.TextNode` objects
        i. **NOTE:** `Docling` is leveraged due to its extensive support with `OCR (Optical Character Recognition` for PDF chunking, but fails to account for other types of documents such ass `.rst` files
4. We then take these chunks and simply add them to our respective `ChromaCollection's`

Proposed Updates - Ingestion

1. Simplifying DataSource's running of Ingestion
    a. No longer using separate Database Transaction, but using same DB that is leveraged in `services/ingestion.py` (no need to commit in `data_providers/base.py` and in `services/file.py`
2. Updating the `cleanup()` function in `FileService` to also go through and *remove Chunks* from ChromaDB that are associated with *stale files*
    a. This can be done by removing nodes from specific Collection by the `file_id` in the metadata
3. Shifting all chunking logic in the `IngestionService` over to the `ChunkingService`
    a. As of now, `ChunkingService` only deals with retrieval, but we should also add the flows surrounding actually chunking code/documentation files
4. When determining file status, we should update the `file.hashed_content` persisted to the database in the case that we've deemed changes have been made
    a. We should also remove all chunks from `ChromaDB` that are associated with this file as their now considered out of date
5. Consolidate our current approach to only leverage *a single ChromaCollection per Project.*
    a. We should remove the separation b/t `documentation` and `code`, and simply store this in the same Chroma collection
6. Expand on the meta data stored in each chunk

    a. Add information such as `content-type` that will display if its code or documentation, or maybe even more granular than that (could be done via heuristic mapping or something like that)
7. Consider leveraging LlamaIndex for Document chunking
    a. Figure out difference between `Docling` vs `LlamaIndex` for PDF parsing (they both you OCR's but one may be better than other)
    b. Using LlamaIndex would allow for us to get the `metadata` for free
8. Store chunks directly in `Postgres` and leverage for BM25
    a. QueryDiffusion that query for direct word *or* semantic similarity
    b. We'll need to remove these records when persisting
9. Consider updating chunking index's to just be UUID

Proposed Updates - RAG

Corrective RAG Approach - High Level

Note: This will also involve updating our `get_chunks()` function to account for

1. Break down user's user into multiple sub-queries
    a. We should have the model determine if we can break this question down to make life a bit easier
2. Leveraging our `Tool` for performing RAG, retrieve the Chunks associated with this sub-query
3. Evaluate the effectiveness of our retrieved Chunks to answer user's question (i.e `RAGAS`)
4. If effective, respond to users question, if not, refine and try to find additional chunks
5. If failed X amount of times, fail safe indicates that unable to answer question

Corrective RAG Approach - Implementation